

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Quiz 1: due 2/25
  - Homework 6: due 2/28
  - Lab 2: due on 2/28
  - Project 2: due on 3/18
- Midterm exam on Thursday 3/3
  - In-person, on paper, closed book

Previous Lecture …

- How to implement condition variables”
- Reflections on using semaphores and condition variables

Muddiest Points (Condition Variable Implementation)

- for the user side implementation of condition variables, is the next queue only holding processes that have signaled and yielded to the awakened process and therefore have to sleep?
- Hoare and Mesa semantics.
- are there any performance drawbacks when implementing condition variables at the application level?
- I think the code for the implementation of the CV was a little fast, so I'm not confident I got it fully.
- How you can connect the cv implementation in and outside kernel
- I don't understand what all the locks do for the condition variable implementation.
- Would you please explain the implementation of Signal() in User-level implement of Condition variable? Thank you.
- Why Hoare semantics are necessary for a user level implementation. The implementation of the user level condition variable was also generally confusing without fully understanding why it's being done that way. I'll need to review this on my own

Muddiest Points (Deadlocks)

- How often do deadlocks occur even if the algorithm tells us it is possible to not deadlock

Muddiest Points (Misc.)

- n/a BUT PLEASE EXTEND PROJ 1. THOTH IS ALWAYSSS DOWNNN!!!!
- where the sleepy barber would actually be used

Final Remarks on Process Synchronization

- Many other synchronization mechanisms
  - Message passing
  - Barriers
  - Futex
  - Re-entrant locks
  - Atomic*

Problem of the Day: CPU Scheduling

How does the ***short-term scheduler*** select the next process to run?

CPU Scheduling

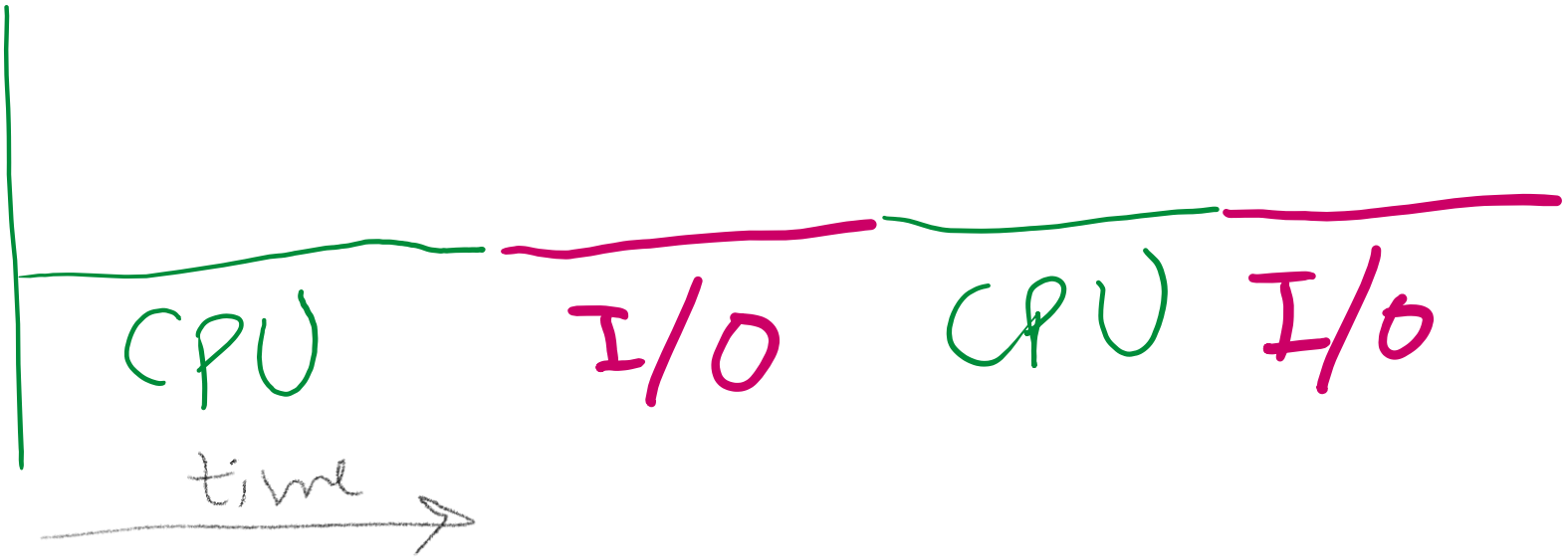
- Scheduling the processor among all ready processes
- User-oriented criteria
  - Response Time: Elapsed time between the submission of a request and the receipt of a response
  - Turnaround Time: Elapsed time between the submission of a process to its completion
- System-oriented criteria
  - Processor utilization
  - Throughput: number of process completed per unit time
  - Fairness

Short-Term Scheduler Dispatcher

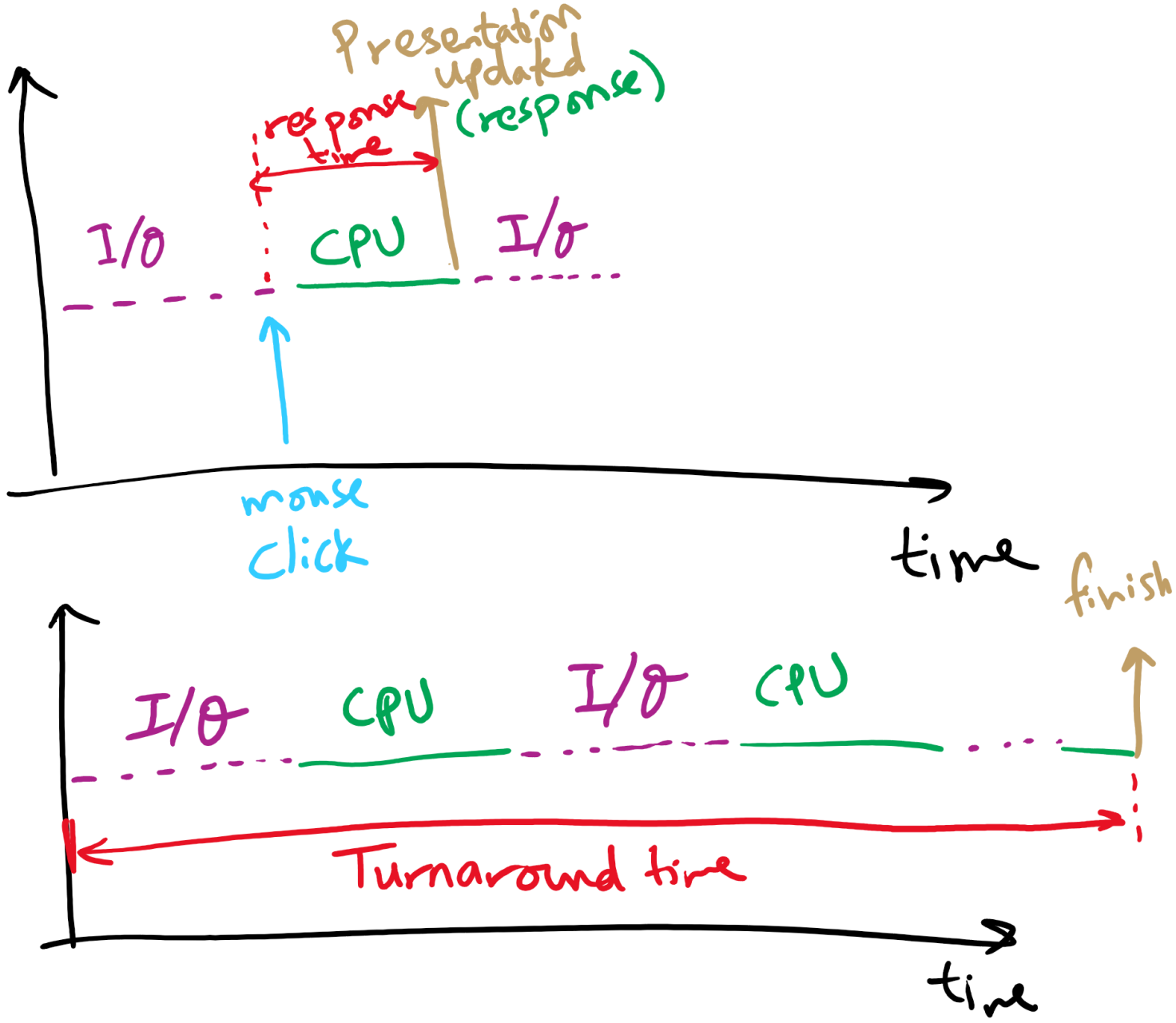
- The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler
- The functions of the dispatcher include:
  - Switching context
  - Switching to user mode
  - Jumping to the location in the user program to restart execution
- The dispatch latency must be minimal

The CPU-I/O Cycle

- Processes require alternate use of processor and I/O in a repetitive fashion
- Each cycle consist of a CPU burst followed by an I/O burst
  - A process terminates on a CPU burst
- CPU-bound processes have longer CPU bursts than I/O-bound processes



Response time vs. Turnaround time



Scheduling Algorithms

- First-Come, First-Served Scheduling
- Shortest-Job-First Scheduling
  - Also referred to as Shortest Process Next
- Priority Scheduling
- Round-Robin Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

Characterization of Scheduling Policies

- The selection function determines which ready process is selected next for execution
- The decision mode specifies the instants in time the selection function is exercised
  - Nonpreemptive
    - Once a process is in the running state, it will continue until it terminates or blocks for an I/O
  - Preemptive
    - Currently running process may be interrupted and moved to the Ready state by the OS
    - Prevents one process from monopolizing the processor

Process Mix Example

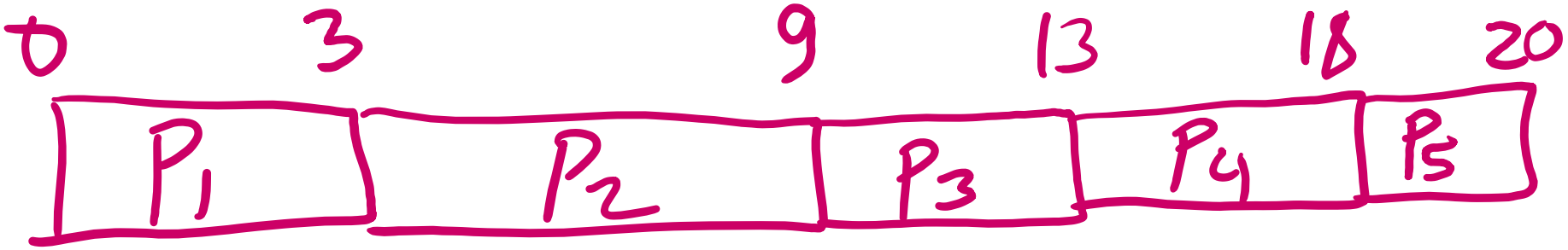
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

Service time = total processor time needed in one (CPU-I/O) cycle
Jobs with long service time are CPU-bound jobs and are referred to as "long jobs"

First Come First Served (FCFS)

- Selection function: the process that has been waiting the longest in the ready queue (hence, FCFS)
- Decision mode: non-preemptive
  - a process runs until it blocks for an I/O

Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



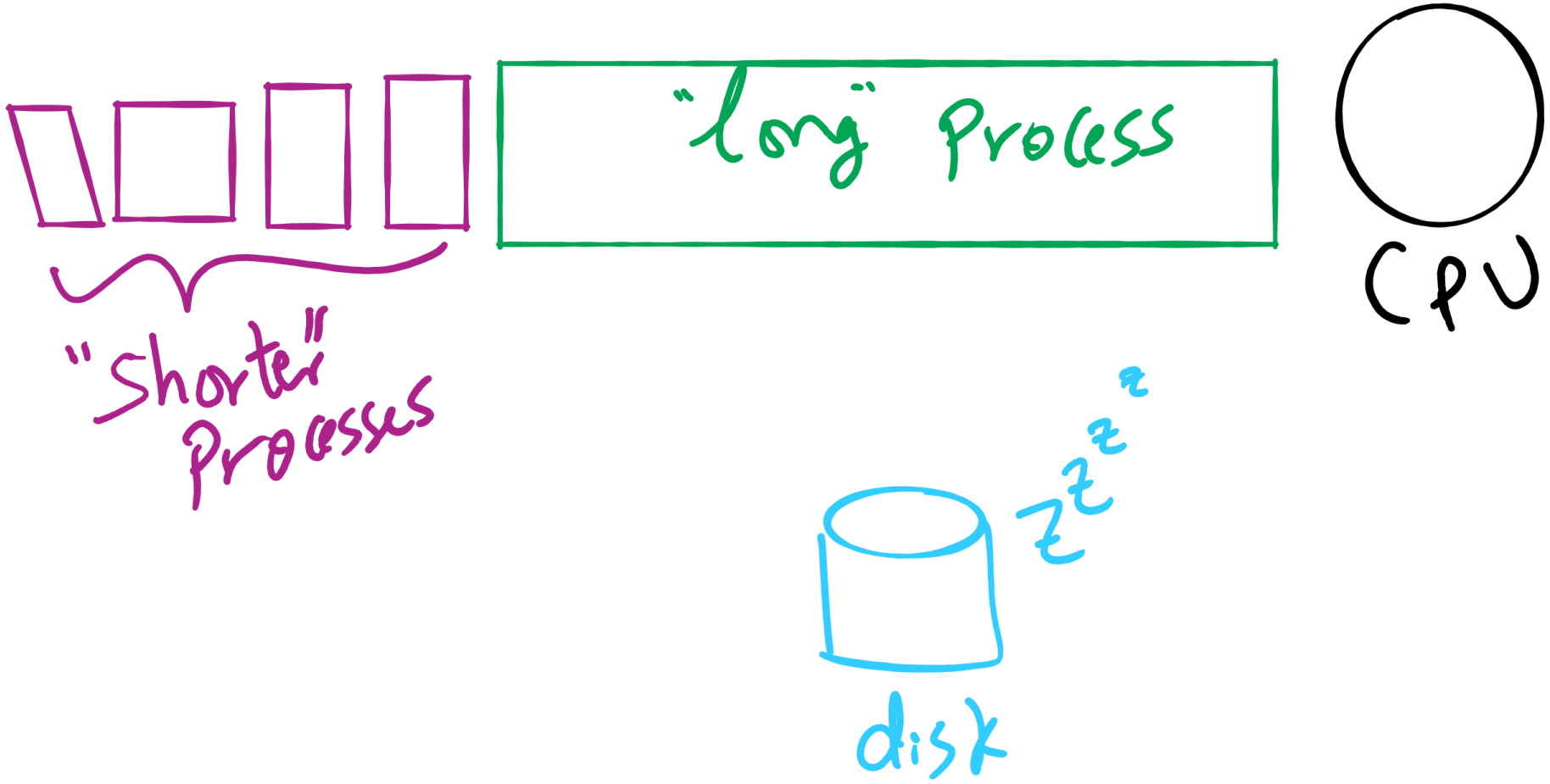
Average Response Time

$$\text{Average Response Time} = \frac{(P_1 - 0) + (P_2 - 2) + (P_3 - 4) + (P_4 - 6) + (P_5 - 8)}{5}$$

FCFS drawbacks

- Favours CPU-bound processes
  - CPU-bound processes monopolize the processor
  - I/O-bound processes have to wait until completion of CPU-bound process
    - I/O-bound processes may have to wait even after their I/Os are completed (poor device utilization)
    - Convoy effect
  - Better I/O device utilization could be achieved if I/O bound processes had higher priority

Convoy Effect



Shortest Job First (Shortest Process Next)

- Selection function: the process with the shortest expected CPU burst time
  - I/O-bound processes will be selected first
- Decision mode: non-preemptive
- The required processing time, i.e., the CPU burst time, must be estimated for each process

Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



SJF / SPN Critique

- Possibility of starvation for longer processes
- Lack of preemption is not suitable in a time sharing environment
- SJF/SPN implicitly incorporates priorities
  - Shortest jobs are given preference
  - CPU bound processes have lower priority, but a process doing no I/O could still monopolize the CPU if it is the first to enter the system

Priorities

- Implemented by having multiple ready queues to represent each level of priority
- Scheduler selects the process of a higher priority over one of lower priority
- Lower-priority may suffer starvation
- To alleviate starvation allow dynamic priorities
  - The priority of a process changes based on its age or execution history

Round-Robin

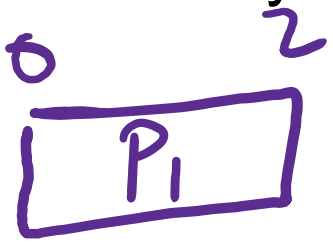
- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

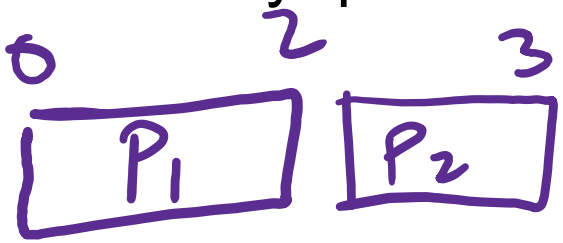
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

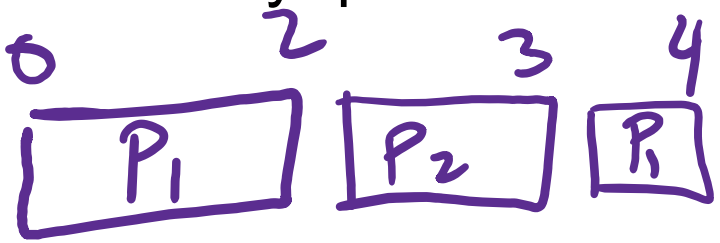
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

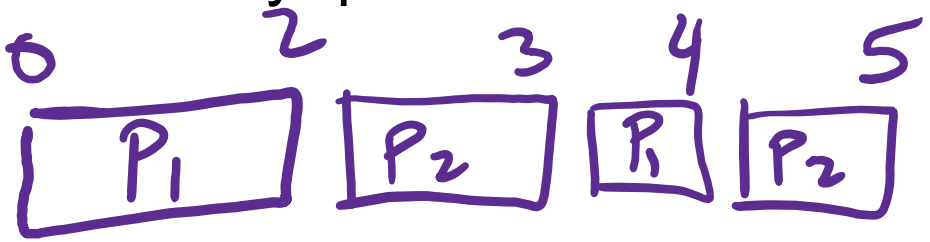
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

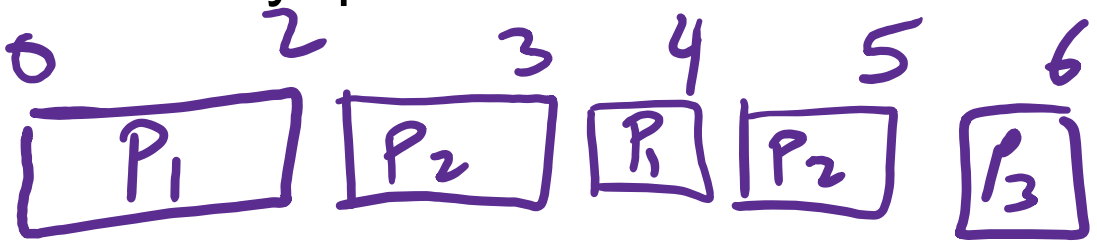
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

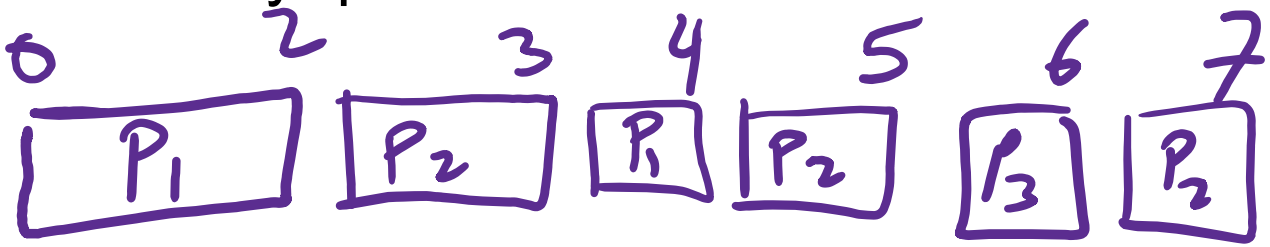
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

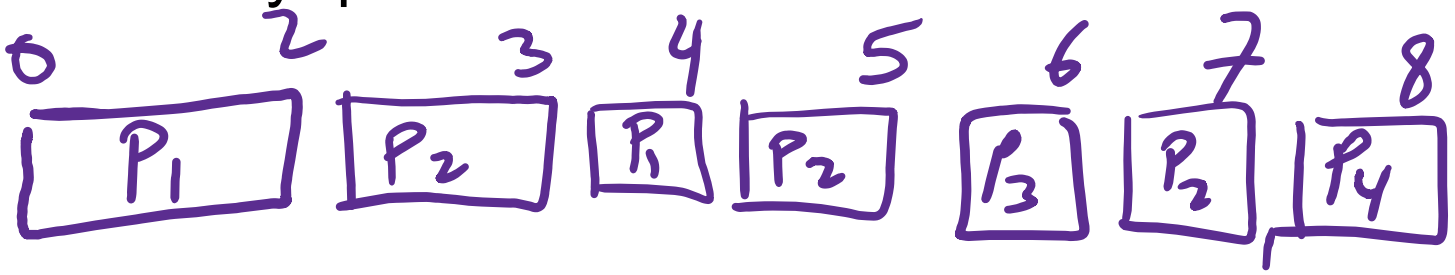
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

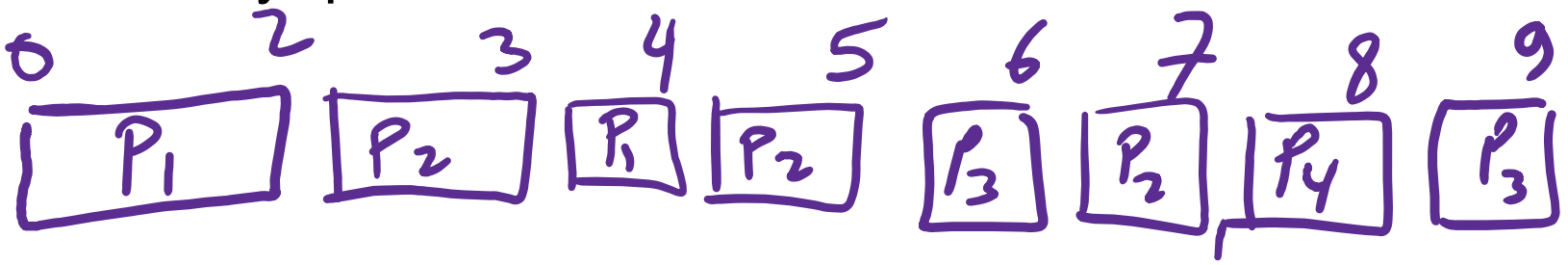
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

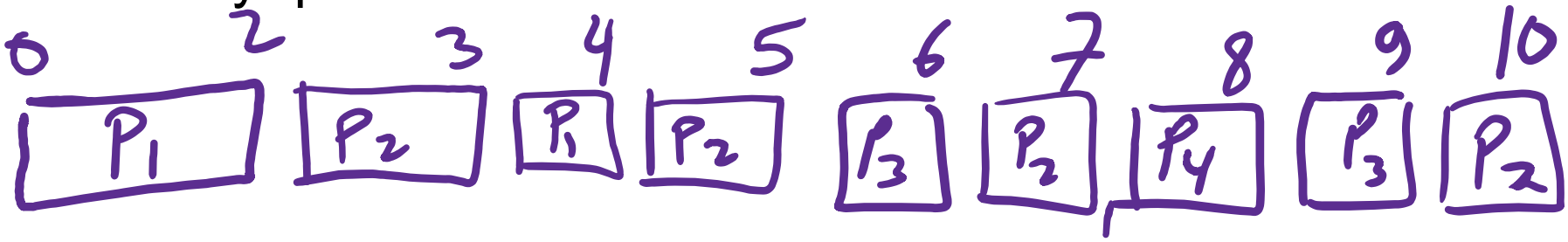
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

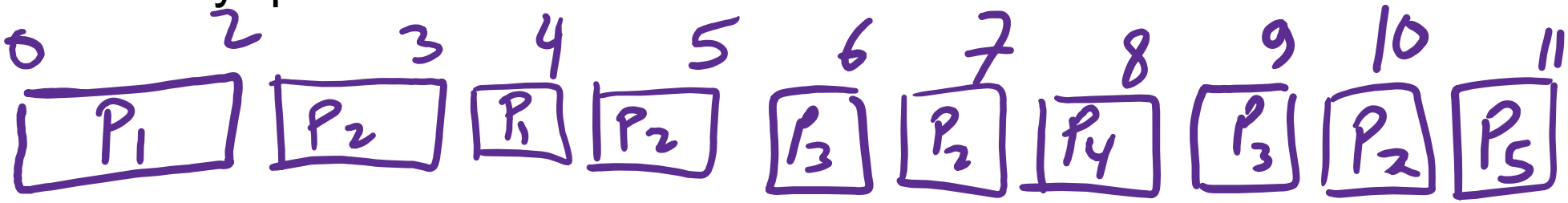
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

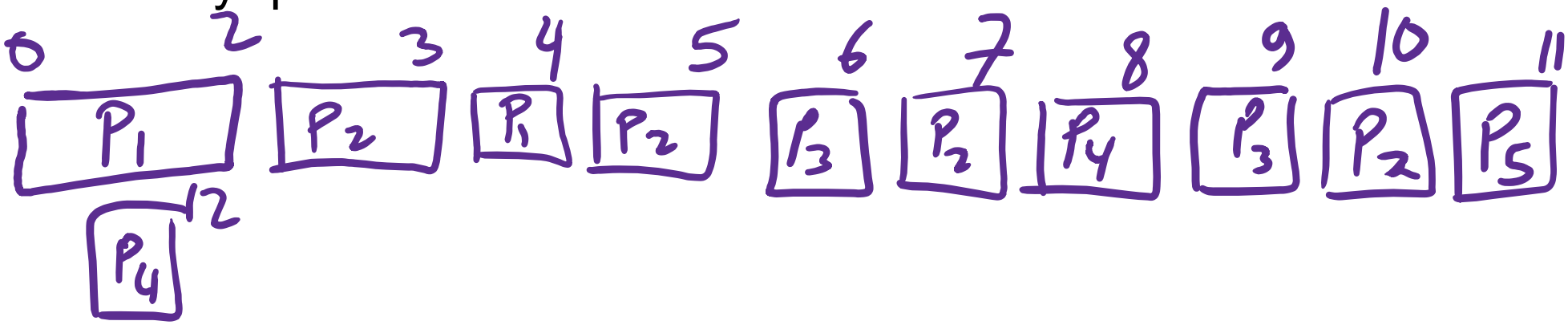
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

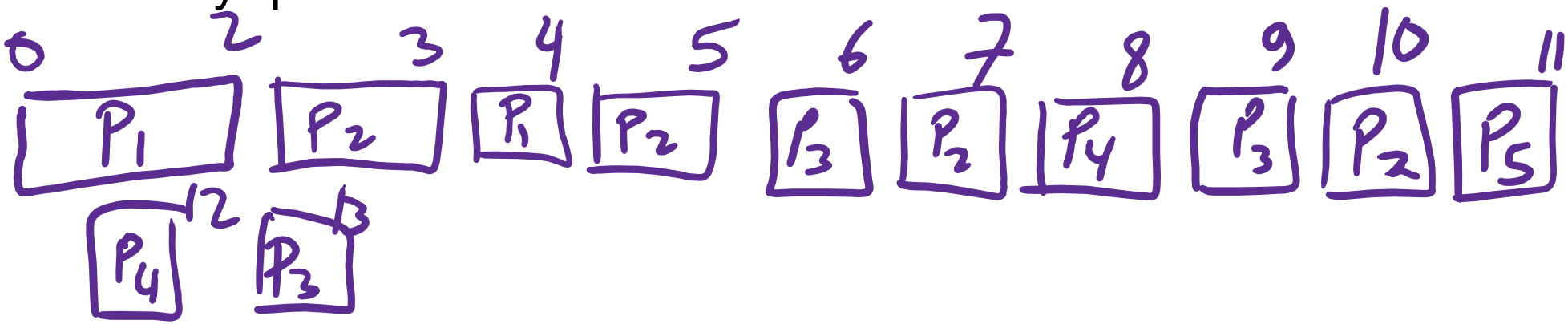
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

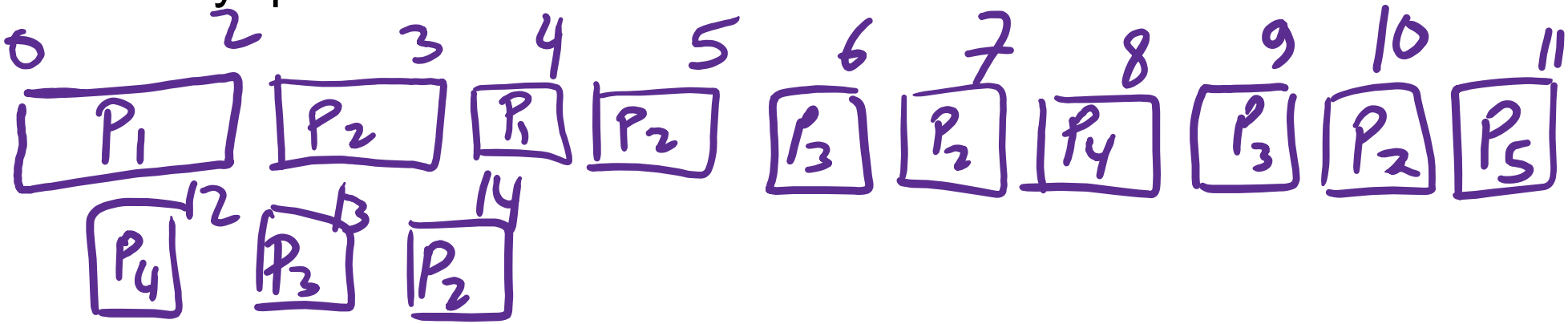
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

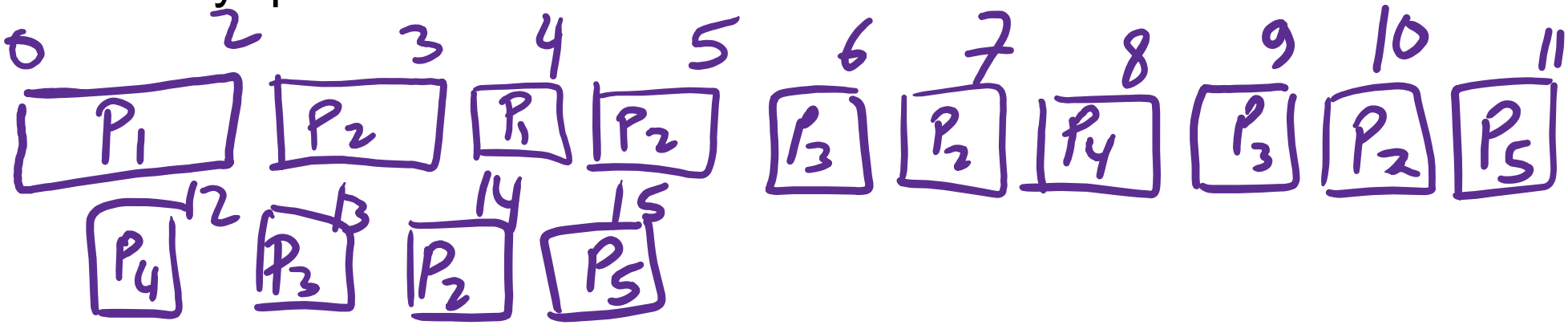
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

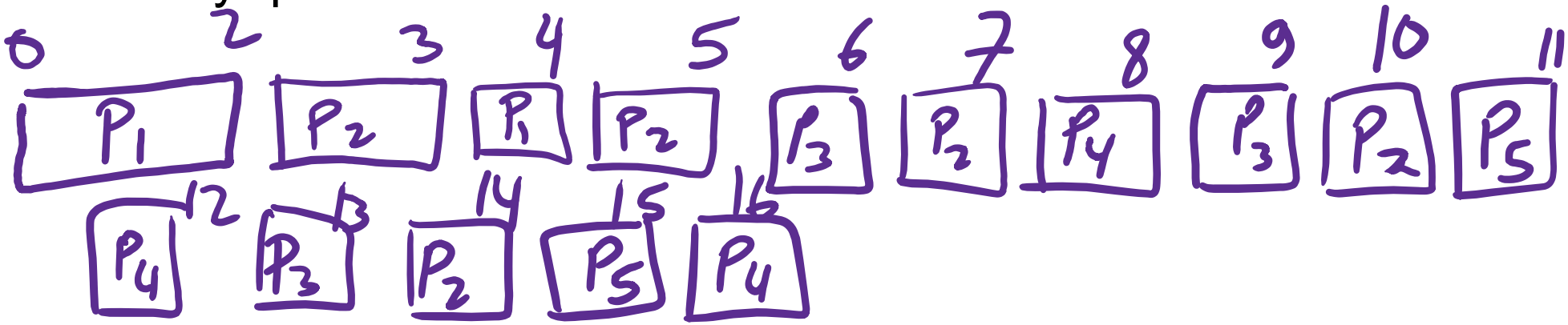
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

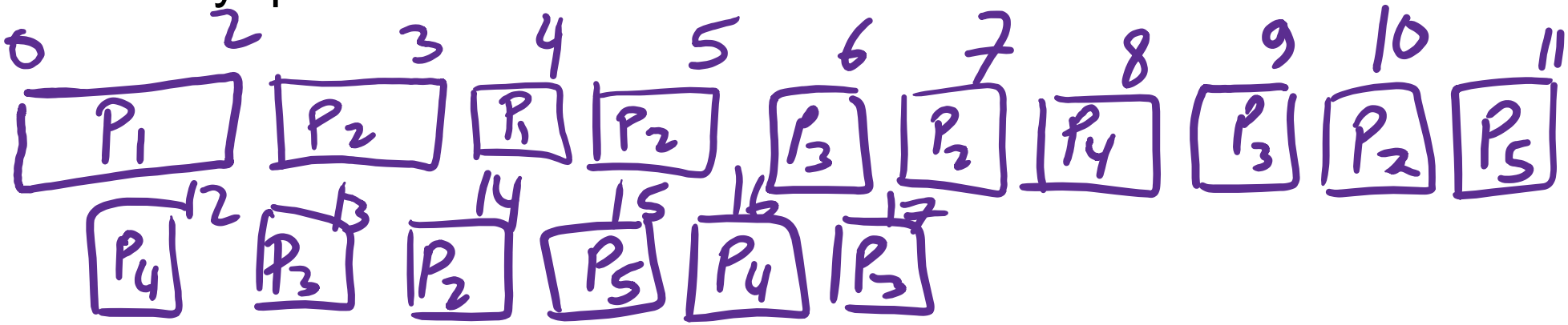
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

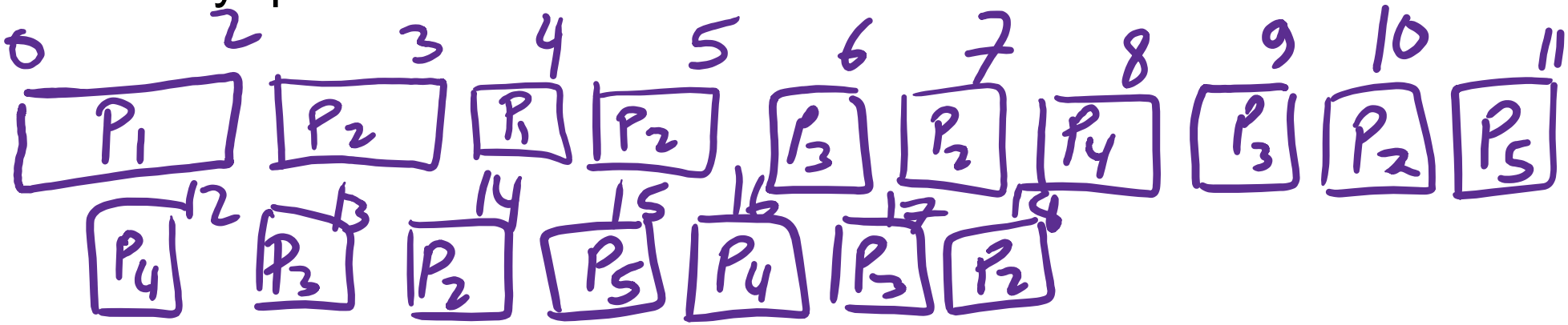
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

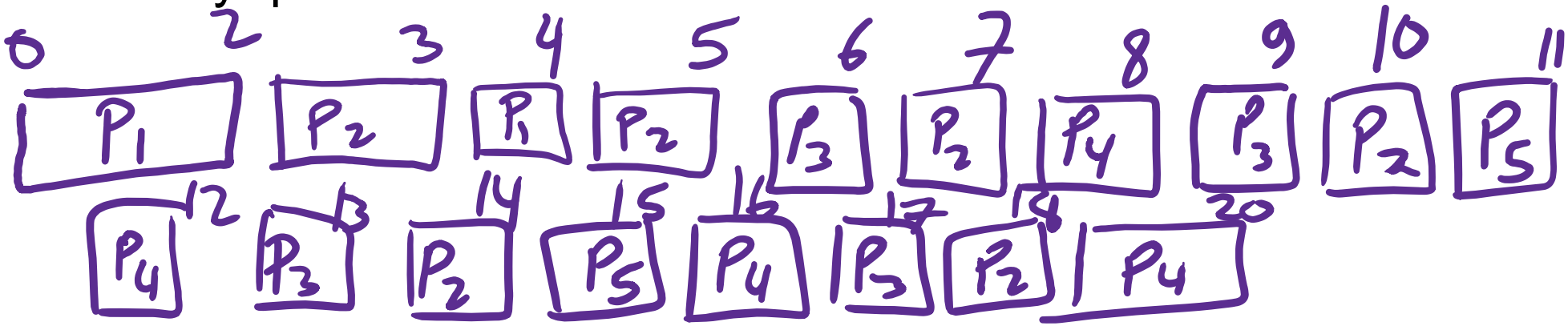
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Round-Robin

- Selection function: same as FCFS
- Decision mode: pre-emptive
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - a clock interrupt occurs and the running process is put on the ready queue

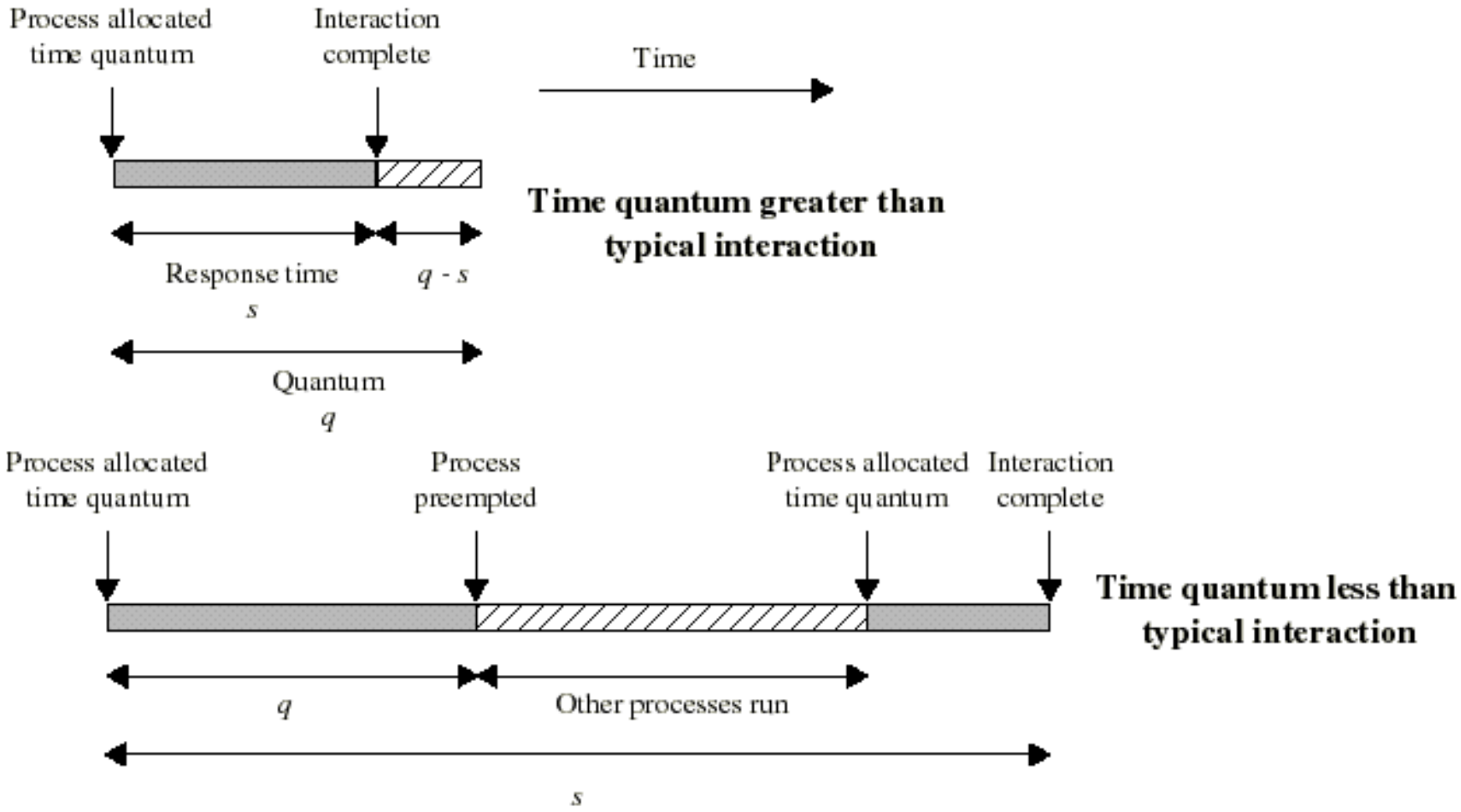
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



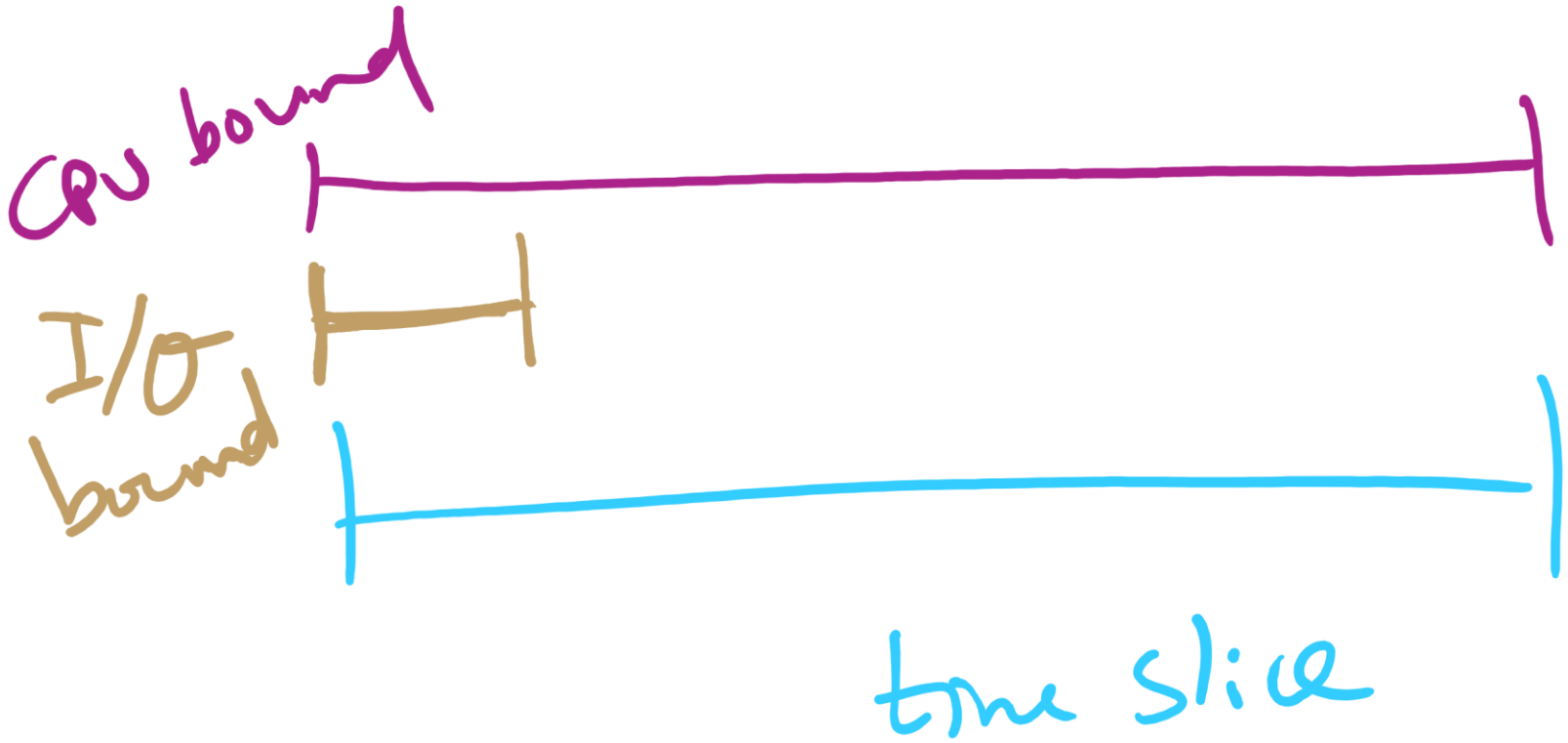
RR Time Quantum

- Quantum must be substantially larger than the time required to handle the clock interrupt and dispatching
- Quantum should be larger then the typical interaction
  - but not much larger, to avoid penalizing I/O bound processes

RR Time Quantum



Quantum Length



Round Robin: critique

- Still favors CPU-bound processes
  - An I/O bound process uses the CPU for a time less than the time quantum before it is blocked waiting for an I/O
  - A CPU-bound process runs for all its time slice and is put back into the ready queue
    - May unfairly get in front of blocked processes